Circom 컴파일러 에러 처리 및 리포팅 시스템 분석

개요

Circom 컴파일러는 포괄적인 에러 처리 및 리포팅 시스템을 통해 개발자에게 명확하고 유용한 에러 정보를 제공합니다. 이 시스템은 `program_structure` 모듈의 `error_definition.rs` 와 `error_code.rs` 를 중심으로 구현되며, 컬러 출력과 소스 코드 하이라이팅을 지원합니다.

1. 에러 분류 체계 및 리포트 생성 분석

1.1 ReportCode 열거형의 에러 분류 체계

Circom 컴파일러는 `ReportCode` 열거형을 통해 체계적인 에러 분류를 제공합니다. 각 에러 코드는 고유한 식별자와 함께 특정 카테고리에 속합니다.

```
// program_structure/src/program_library/error_code.rs

#[derive(Copy,Clone)]

pub  enum ReportCode {

// 파싱 에러 (P1xxx)

UnclosedComment, // P1005

FileOs, // P1006

NoMainFoundInProject, // P1001

MultipleMain, // P1002

MissingSemicolon, // P1008

UnrecognizedInclude, // P1009
```

```
UnrecognizedVersion, // P1010
UnrecognizedPragma, // P1011
IllegalExpression, // P1012
MultiplePragma, // P1013
IncludeNotFound, // P1014
ExpectedIdentifier, // P1015
// 타입 분석 에러 (T2xxx)
UndefinedFunction, // T2001
UndefinedTemplate, // T2002
UninitializedSymbolInExpression, // T2003
UnableToTypeFunction, // T2004
UnreachableConstraints, // T2005
SameFunctionDeclaredTwice, // T2006
SameTemplateDeclaredTwice, // T2007
SameSymbolDeclaredTwice, // T2008
// 런타임 에러 (T3xxx)
RuntimeError, // T3001
RuntimeWarning, // T3002
// 제약 조건 분석 에러 (CAxx)
UnconstrainedSignal, // CA01
UnconstrainedIOSignal, // CA02
UnusedInput, // CA03
UnusedOutput, // CA04
```

```
// 커스텀 게이트 에러 (CGxx)
CustomGateIntermediateSignalWarning, // CG01
CustomGateConstraintError, // CG02
CustomGateSubComponentError, // CG03
CustomGatesPragmaError, // CG04
CustomGatesVersionError, // CG05
// 버스 관련 에러 (BUxx)
BusWrongNumberOfArguments, // BU01
InvalidArgumentInBusInstantiationT, // BU02
InvalidArgumentInBusInstantiationB, // BU03
InvalidSignalAccessInBus, // BU04
MustBeSameBus, // BU05
MustBeBus, // BU06
}
```

에러 코드 체계의 특징

1. **계층적 분류**: 에러 코드는 접두사를 통해 카테고리별로 분류됩니다

- `P1xxx` : 파싱 단계 에러
- `T2xxx` : 타입 분석 단계 에러
- `T3xxx` : 런타임 에러
- `CAxx` : 제약 조건 분석 에러
- `CGxx` : 커스텀 게이트 관련 에러
- `BUxx` : 버스 관련 에러

2. **매개변수화된 에러**: 일부 에러는 추가 정보를 포함합니다

```
WrongTypesInAssignOperationDims(usize, usize), // 예상 차원, 실제 차원
WrongNumberOfArguments(usize, usize), // 예상 인수, 실제 인수
NonHomogeneousArray(usize, usize), // 예상 차원, 실제 차원
InvalidArrayAccess(usize, usize), // 배열 크기, 접근 인덱스
```

3. **Display 트레이트 구현**: 각 에러 코드는 고유한 문자열 식별자로 변환됩니다

```
impl fmt::Display for ReportCode {
fn  fmt(&self, f: &mut Formatter<'_>) -> fmt::Result {
let  string_format = match  self {
NoMainFoundInProject => "P1001",
MultipleMain => "P1002",
CompilerVersionError => "P1003",
// ... 기타 매핑
};
f.write_str(string_format)
}
}
```

1.2 Report 구조체의 에러 정보 수집 및 포맷팅

`Report` 구조체는 에러 정보를 수집하고 포맷팅하는 핵심 데이터 구조입니다.

```rust
// program_structure/src/program_library/error_definition.rs

#[derive(Clone)]

pub struct Report {

category: MessageCategory, // Error 또는 Warning

error_message: String, // 에러 메시지

error_code: ReportCode, // 에러 코드

primary: Vec<ReportLabel>, // 주요 위치 정보

secondary: Vec<ReportLabel>, // 보조 위치 정보

notes: Vec<ReportNote>, // 추가 노트

}
```

MessageCategory 열거형

```rust
#[derive(Copy, Clone)]

pub enum MessageCategory {

Error,

Warning,

}


impl MessageCategory {

fn is_error(&self) -> bool {

matches!(self, MessageCategory::Error)

}
```

```
fn is_warning(&self) -> bool {
matches!(self, MessageCategory::Warning)
}
}
```

Report 생성 및 조작 메서드

1. **기본 생성자**:

```
pub fn error(error_message: String, code: ReportCode) -> Report
pub fn warning(error_message: String, code: ReportCode) -> Report
```

2. **위치 정보 추가**:

```
pub fn add_primary(
&mut self,
location: FileLocation,
file_id: FileID,
message: String,
) -> &mut Self
pub fn add_secondary(
&mut self,
location: FileLocation,
file_id: FileID,
possible_message: Option<String>,
```

```
) -> &mut Self
```

3. **추가 정보 첨부**:

```
pub fn add_note(&mut self, note: String) -> &mut Self
```

실제 사용 예시

```
// type_analysis 모듈에서의 에러 생성 예시
let mut report = Report::error(
"Signal, bus or component declaration inside While scope".to_string(),
ReportCode::SignalOutsideOriginalScope,
);
report.add_primary(
meta.file_location(),
meta.get_file_id(),
"Signal declared here".to_string()
);
report.add_note(
"Signals can only be defined in the initial scope or in If scopes with known con
);
```

1.3 다국어 에러 메시지 및 컬러 출력 시스템

컬러 출력 시스템

Circom 컴파일러는 `codespan_reporting` 크레이트를 사용하여 컬러 출력을 지원합니다:

```
pub fn print_reports(reports: &[Report], file_library: &FileLibrary) {
use codespan_reporting::term::termcolor::{ColorChoice, StandardStream};
let writer = StandardStream::stderr(ColorChoice::Always);
let config = term::Config::default();
let mut diagnostics = Vec::new();
let files = file_library.to_storage();
// Report를 Diagnostic으로 변환
for report in reports.iter() {
diagnostics.push(report.to_diagnostic());
}
// 각 진단 정보를 컬러로 출력
for diagnostic in diagnostics.iter() {
let print_result = term::emit(&mut writer.lock(), &config, files, &diagnostic)
if print_result.is_err() {
panic!("Error printing reports")
}
}
}
```

Diagnostic 변환 과정

```
pub fn to_diagnostic(&self) -> Diagnostic<FileID> {
let mut labels = self.get_primary().clone();
let mut secondary = self.get_secondary().clone();
labels.append(&mut secondary);


if self.is_warning() {
Diagnostic::warning()
} else {
Diagnostic::error()
}
.with_message(self.get_message())
.with_code(Report::error_code_to_diagnostic_code(self.get_code()))
.with_labels(labels)
.with_notes(self.get_notes().clone())
}
```

에러 코드 변환

```
pub fn error_code_to_diagnostic_code(error_code: &ReportCode) -> DiagnosticCod
error_code.to_string() // ReportCode의 Display 구현 사용
}
```

출력 예시

컬러 출력 시스템은 다음과 같은 형태로 에러를 표시합니다:

```
error[P1001]: No main component found in project

--> circuit.circom:15:1

|

15 | template Multiplier() {

| ^^^^^^^^^^^^^^^^^^^^^^ main component should be declared here

|

= note: Every Circom project must have exactly one main component
```

다국어 지원 고려사항

현재 Circom 컴파일러는 영어 메시지를 기본으로 하지만, 다국어 지원을 위한 확장 가능한 구조를 가지고 있습니다:

1. **메시지 외부화**: 에러 메시지를 별도 파일로 분리 가능
2. **로케일 감지**: 시스템 로케일에 따른 메시지 선택
3. **템플릿 시스템**: 매개변수화된 메시지의 다국어 지원

```
// 향후 다국어 지원을 위한 구조 예시

pub struct LocalizedMessage {

code: ReportCode,

locale: String,

template: String,

parameters: Vec<String>,

}
```

이러한 에러 처리 시스템은 개발자가 Circom 코드의 문제점을 빠르게 파악하고 수정할 수 있도록 도와주는 핵심 기능입니다.

2.

소스 코드 위치 추적 및 하이라이팅 분석

2.1 FileLocation 및 Meta 구조체의 위치 정보 관리

Circom 컴파일러는 정확한 에러 위치 추적을 위해 `FileLocation` 과 `Meta` 구조체를 사용합니다.

FileLocation 타입 정의

```
// program_structure/src/program_library/file_definition.rs
use std::ops::Range;

pub type FileSource = String;
pub type FilePath = String;
pub type FileID = usize;
pub type FileLocation = Range<usize>; // 시작과 끝 바이트 위치
```

`FileLocation` 은 소스 코드 내에서 특정 구문의 시작과 끝 위치를 바이트 단위로 나타내는 범위(Range)입니다.

Meta 구조체의 위치 정보 관리

```
// program_structure/src/abstract_syntax_tree/ast.rs
#[derive(Clone)]
pub struct Meta {
```

```
pub elem_id: usize, // 요소 고유 ID
pub start: usize, // 시작 위치 (호환성용)
pub end: usize, // 끝 위치 (호환성용)
pub location: FileLocation, // 실제 파일 위치 (Range<usize>)
pub file_id: Option<usize>, // 파일 식별자
pub component_inference: Option<String>, // 컴포넌트 추론 정보
type_knowledge: TypeKnowledge, // 타입 정보
memory_knowledge: MemoryKnowledge, // 메모리 정보
}
```

Meta 구조체의 핵심 메서드

1. **위치 정보 접근**:

```
impl Meta {
pub fn get_start(&self) -> usize {
self.location.start
}
pub fn get_end(&self) -> usize {
self.location.end
}
pub fn get_file_id(&self) -> usize {
if let Option::Some(id) = self.file_id {
id
```

```
} else {
panic!("Empty file id accessed")
}
}
pub fn file_location(&self) -> FileLocation {
self.location.clone()
}
}
```

2. **위치 정보 설정**:

```
pub fn change_location(&mut self, location: FileLocation, file_id: Option<usi
self.location = location;
self.file_id = file_id;
}
pub fn set_file_id(&mut self, file_id: usize) {
self.file_id = Option::Some(file_id);
}
```

FileLibrary를 통한 파일 관리

```
// program_structure/src/program_library/file_definition.rs
#[derive(Clone)]
pub struct FileLibrary {
```

```
files: FileStorage, // SimpleFiles<FilePath, FileSource>
}

impl FileLibrary {
pub fn add_file(&mut self, file_name: FilePath, file_source: FileSource) -> F
self.get_mut_files().add(file_name, file_source)
}
pub fn get_line(&self, start: usize, file_id: FileID) -> Option<usize> {
match self.files.line_index(file_id, start) {
Some(lines) => Some(lines + 1), // 1-based 라인 번호
None => None,
}
}
pub fn to_storage(&self) -> &FileStorage {
&self.get_files()
}
}
```

2.2 에러 발생 지점의 소스 코드 하이라이팅 및 컨텍스트 표시

ReportLabel을 통한 위치 표시

Circom 컴파일러는 `codespan_reporting` 크레이트의 `Label` 시스템을 사용하여 소스 코드 하이라이팅을 구현합니다:

```rust
// program_structure/src/program_library/error_definition.rs

type ReportLabel = Label<FileID>;



impl Report {

pub fn add_primary(

&mut self,

location: FileLocation,

file_id: FileID,

message: String,

) -> &mut Self {

let label = ReportLabel::primary(file_id, location).with_message(message);

self.get_mut_primary().push(label);

self

}

pub fn add_secondary(

&mut self,

location: FileLocation,

file_id: FileID,

possible_message: Option<String>,

) -> &mut Self {

let mut label = ReportLabel::secondary(file_id, location);

if let Option::Some(message) = possible_message {
```

```
label = label.with_message(message);
}
self.get_mut_secondary().push(label);
self
}
}
```

Primary vs Secondary 라벨의 차이점

1. **Primary Label**: 주요 에러 발생 지점
- 빨간색으로 하이라이팅
- 에러의 직접적인 원인이 되는 코드 위치
- 예: 잘못된 타입의 변수 할당
2. **Secondary Label**: 관련된 보조 정보
- 파란색으로 하이라이팅
- 에러와 관련된 추가 컨텍스트 제공
- 예: 이전에 선언된 변수의 위치

실제 에러 리포트 생성 예시

```
// 중복 심볼 선언 에러 예시
let mut report = Report::error(
String::from("Duplicated callable symbol"),
ReportCode::SameSymbolDeclaredTwice,
);
```

```
// 주요 에러 위치 (새로운 선언)
report.add_primary(
current_meta.file_location(),
current_file_id,
format!("{} is already in use", symbol_name)
);

// 보조 정보 (이전 선언 위치)
report.add_secondary(
previous_meta.file_location(),
previous_file_id,
Some("Previously declared here".to_string())
);
```

신호 할당 에러의 복합 위치 추적 예시

```
// constraint_generation/src/execute.rs에서의 예시
let mut rep = Report::error(
format!("Exception caused by invalid assignment\n Signal has been already assign
RuntimeError
);

// 현재 할당 시도 위치 (Primary)
```

```
rep.add_primary(
current_meta.file_location(),
current_meta.get_file_id(),
"Invalid assignment here".to_string()
);


// 이전 할당 위치 (Secondary)
rep.add_secondary(
previous_meta.file_location(),
previous_meta.get_file_id(),
Option::Some("This is the previous assignment to the variable".to_string())
);
```

2.3 다중 파일 프로젝트에서의 에러 추적 메커니즘

Include 파일 처리 시 에러 추적

Circom 프로젝트는 여러 `.circom` 파일로 구성될 수 있으며, 각 파일은 고유한 `FileID` 를 가집니다:

```
// parser/src/lib.rs에서 다중 메인 컴포넌트 에러 처리
let mut r = Report::error(
"Multiple main components in the project".to_string(),
ReportCode::MultipleMain
);
```

```
for (file_id, expression, _) in  main_components {
if  j > 0 {
// 추가 메인 컴포넌트들을 Secondary로 표시
r.add_secondary(
expression.get_meta().location.clone(),
file_id,
Option::Some("Here it is another main component".to_string())
);
} else {
// 첫 번째 메인 컴포넌트를 Primary로 표시
r.add_primary(
expression.get_meta().location.clone(),
file_id,
"This is a main component".to_string()
);
}
j += 1;
}
```

파일 간 의존성 에러 추적

Include 관계에서 발생하는 에러는 여러 파일에 걸쳐 추적됩니다:

```
// Include 파일을 찾을 수 없는 경우
```

```
Result::Err(produce_report_with_message(
ReportCode::IncludeNotFound,
format!("Cannot find include file: {}", include_path)
))


// 순환 의존성 에러의 경우 전체 경로를 표시
let  path_display = IncludesGraph::display_path(circular_path);
Report::error(
format!("Circular dependency detected: {}", path_display),
ReportCode::CustomGatesPragmaError
)
```

컨텍스트 정보 제공

에러 리포트는 다음과 같은 컨텍스트 정보를 제공합니다:

1. **파일 경로**: 에러가 발생한 정확한 파일
2. **라인 번호**: `FileLibrary::get_line()` 메서드를 통한 라인 정보
3. **컬럼 위치**: 바이트 오프셋을 통한 정확한 위치
4. **소스 코드 스니펫**: 에러 주변의 코드 컨텍스트
5. **관련 위치들**: Primary와 Secondary 라벨을 통한 연관 위치

출력 예시

다중 파일 프로젝트에서의 에러 출력 예시:

```
error[T2008]: Duplicated callable symbol
```

```
--> src/main.circom:15:10

|

15 | template Multiplier() {

| ^^^^^^^^^^^ `Multiplier` is already in use

|

note: Previously declared here

--> src/utils.circom:8:10

|

8 | template Multiplier(n) {

| ^^^^^^^^^^

|

= help: Consider renaming one of the templates
```

이러한 위치 추적 시스템을 통해 개발자는 복잡한 다중 파일 프로젝트에서도 에러의 정확한 위치와 원인을 빠르게 파악할 수 있습니다.

2.4 에러 추적의 성능 최적화

지연 평가 (Lazy Evaluation)

```rust
// 에러가 발생했을 때만 위치 정보를 계산

pub fn get_line_lazy(&self, start: usize, file_id: FileID) -> Option<usize> {

// 실제 라인 계산은 에러 출력 시점에 수행

self.files.line_index(file_id, start).map(|line| line + 1)

}
```

메모리 효율적인 위치 저장

```
// Range<usize>를 사용하여 메모리 효율성 확보

pub  type FileLocation = Range<usize>; // 16바이트 (64비트 시스템)



// 대신 별도의 구조체를 사용하지 않음

// struct Location { start: usize, end: usize, line: usize, column: usize } // 3
```

이러한 에러 처리 및 리포팅 시스템은 Circom 컴파일러의 사용성을 크게 향상시키며, 개발자가 복잡한 zkSNARK 회로를 효율적으로 개발할 수 있도록 지원합니다.